

08_full_panel_text_indicator_analysis

May 11, 2026

1 08 Full Panel + Text Indicators (Descriptive)

This notebook builds the first full monthly analysis panel by merging: - combined polling + media salience panel (2013-09 to 2025-12), and - monthly text/content indicators (2013-01 to 2025-12).

Scope: - Descriptive reporting only. - No causal claims.

```
[1]: from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

pd.set_option('display.max_columns', 300)
pd.set_option('display.width', 220)

PROJECT_ROOT = Path.cwd()
if PROJECT_ROOT.name == 'notebooks':
    PROJECT_ROOT = PROJECT_ROOT.parent

PANEL_DIR = PROJECT_ROOT / 'data' / 'panel'
INDICATORS_DIR = PROJECT_ROOT / 'data' / 'indicators'
FIGURES_DIR = PROJECT_ROOT / 'figures' / '08_notebook_figures'

FIGURES_DIR.mkdir(parents=True, exist_ok=True)

COMBINED_PANEL_PARQUET = PANEL_DIR / \
    'monthly_salience_polling_panel_combined_2013_2025.parquet'
COMBINED_PANEL_CSV = PANEL_DIR / \
    'monthly_salience_polling_panel_combined_2013_2025.csv'

TEXT_MONTHLY_PARQUET = INDICATORS_DIR / 'monthly_text_content_indicators.'
    'parquet'
TEXT_MONTHLY_CSV = INDICATORS_DIR / 'monthly_text_content_indicators.csv'

OUT_FULL_PANEL_CSV = PANEL_DIR / 'monthly_full_analysis_panel_2013_2025.csv'
OUT_FULL_PANEL_PARQUET = PANEL_DIR / 'monthly_full_analysis_panel_2013_2025.'
    'parquet'
```

```

OUT_SHOCK_SUMMARY_CSV = PANEL_DIR / 'full_panel_text_shock_window_summary.csv'

EXPECTED_COMBINED_RANGE = pd.period_range('2013-09', '2025-12', freq='M')
EXPECTED_TEXT_RANGE = pd.period_range('2013-01', '2025-12', freq='M')
EXPECTED_MERGED_RANGE = pd.period_range('2013-09', '2025-12', freq='M')
EXPECTED_MERGED_ROWS = len(EXPECTED_MERGED_RANGE)

SHOCKS = {
    'Cologne': pd.Timestamp('2016-01-01'),
    'Chemnitz': pd.Timestamp('2018-08-01'),
    'Correctiv': pd.Timestamp('2024-01-01'),
}

KEY_TEXT_COLS = [
    'migration_terms_per_1000_mean',
    'security_terms_per_1000_mean',
    'afd_far_right_terms_per_1000_mean',
    'remigration_democracy_terms_per_1000_mean',
    'negative_threat_terms_per_1000_mean',
    'n_articles_text_indicators',
]

print('COMBINED_PANEL_PARQUET:', COMBINED_PANEL_PARQUET)
print('TEXT_MONTHLY_PARQUET:', TEXT_MONTHLY_PARQUET)
print('OUT_FULL_PANEL_CSV:', OUT_FULL_PANEL_CSV)
print('FIGURES_DIR:', FIGURES_DIR)

```

```

COMBINED_PANEL_PARQUET: c:\PythonProjects\AfD-
project\data\panel\monthly_salience_polling_panel_combined_2013_2025.parquet
TEXT_MONTHLY_PARQUET: c:\PythonProjects\AfD-
project\data\indicators\monthly_text_content_indicators.parquet
OUT_FULL_PANEL_CSV: c:\PythonProjects\AfD-
project\data\panel\monthly_full_analysis_panel_2013_2025.csv
FIGURES_DIR: c:\PythonProjects\AfD-project\figures\08_notebook_figures

```

```

[2]: def load_table(parquet_path: Path, csv_path: Path, label: str) -> pd.DataFrame:
    if parquet_path.exists():
        df = pd.read_parquet(parquet_path)
        print(f'Loaded {label} from parquet: {parquet_path}')
        return df
    if csv_path.exists():
        df = pd.read_csv(csv_path)
        print(f'Loaded {label} from csv: {csv_path}')
        return df
    raise FileNotFoundError(f'Missing {label} input files: {parquet_path} and_
↪ {csv_path}')

```

```

def ensure_month_keys(df: pd.DataFrame, label: str) -> pd.DataFrame:
    out = df.copy()

    if 'year_month' in out.columns:
        ym = out['year_month'].astype(str).str.strip()
    elif 'month' in out.columns:
        ym = out['month'].astype(str).str.strip()
    elif 'month_dt' in out.columns:
        dt = pd.to_datetime(out['month_dt'], errors='coerce')
        if dt.isna().any():
            raise ValueError(f'{label}: month_dt contains unparseable values')
        ym = dt.dt.strftime('%Y-%m')
    else:
        raise ValueError(f'{label}: cannot derive month key (need year_month/
        month/month_dt)')

    dt_from_ym = pd.to_datetime(ym + '-01', errors='coerce')
    if dt_from_ym.isna().any():
        bad = ym[dt_from_ym.isna()].head(10).tolist()
        raise ValueError(f'{label}: unparseable month values: {bad}')

    out['year_month'] = dt_from_ym.dt.strftime('%Y-%m')
    out['month_dt'] = dt_from_ym
    out['month'] = out['year_month']
    out['year'] = out['month_dt'].dt.year
    out['month_num'] = out['month_dt'].dt.month
    return out

combined_raw = load_table(COMBINED_PANEL_PARQUET, COMBINED_PANEL_CSV, 'combined_
panel')
text_raw = load_table(TEXT_MONTHLY_PARQUET, TEXT_MONTHLY_CSV, 'monthly text_
indicators')

combined = ensure_month_keys(combined_raw, 'combined panel')
text_monthly = ensure_month_keys(text_raw, 'monthly text indicators')

print('combined shape:', combined.shape)
print('text_monthly shape:', text_monthly.shape)

```

Loaded combined panel from parquet: c:\PythonProjects\AfD-
 project\data\panel\monthly_salience_polling_panel_combined_2013_2025.parquet
 Loaded monthly text indicators from parquet: c:\PythonProjects\AfD-
 project\data\indicators\monthly_text_content_indicators.parquet
 combined shape: (148, 41)
 text_monthly shape: (156, 31)

```

[3]: required_text_cols = KEY_TEXT_COLS.copy()
missing_text_cols = [c for c in required_text_cols if c not in text_monthly.
    ↪columns]
if missing_text_cols:
    raise ValueError(f'Missing required text indicator columns: {
    ↪missing_text_cols}')

if combined['year_month'].duplicated().any():
    dupes = combined.loc[combined['year_month'].duplicated(), 'year_month'].
    ↪tolist()
    raise ValueError(f'Combined panel has duplicate months: {dupes[:10]}')

if text_monthly['year_month'].duplicated().any():
    dupes = text_monthly.loc[text_monthly['year_month'].duplicated(),
    ↪'year_month'].tolist()
    raise ValueError(f'Text indicators have duplicate months: {dupes[:10]}')

combined_periods = pd.PeriodIndex(combined['year_month'], freq='M')
text_periods = pd.PeriodIndex(text_monthly['year_month'], freq='M')

missing_combined_months = EXPECTED_COMBINED_RANGE.difference(combined_periods)
extra_combined_months = combined_periods.difference(EXPECTED_COMBINED_RANGE)
missing_text_months = EXPECTED_TEXT_RANGE.difference(text_periods)
extra_text_months = text_periods.difference(EXPECTED_TEXT_RANGE)

if len(missing_combined_months) > 0 or len(extra_combined_months) > 0:
    raise ValueError(
        'Combined panel coverage mismatch. '
        f'missing={ [p.strftime("%Y-%m") for p in missing_combined_months] }, '
        f'extra={ [p.strftime("%Y-%m") for p in extra_combined_months] }'
    )

if len(missing_text_months) > 0 or len(extra_text_months) > 0:
    raise ValueError(
        'Text indicator coverage mismatch. '
        f'missing={ [p.strftime("%Y-%m") for p in missing_text_months] }, '
        f'extra={ [p.strftime("%Y-%m") for p in extra_text_months] }'
    )

print('Validation summary before merge:')
print('combined months:', combined['year_month'].min(), 'to',
    ↪combined['year_month'].max(), 'rows=', len(combined))
print('text months:', text_monthly['year_month'].min(), 'to',
    ↪text_monthly['year_month'].max(), 'rows=', len(text_monthly))

```

Validation summary before merge:
combined months: 2013-09 to 2025-12 rows= 148

text months: 2013-01 to 2025-12 rows= 156

```
[4]: text_merge_cols = [
    'year_month',
    'text_indicator_month_has_articles',
    'n_articles_text_indicators',
    'mean_word_count',
    'median_word_count',
    'total_word_count',
    'migration_terms_per_1000_mean',
    'migration_terms_total',
    'share_articles_with_migration_terms',
    'security_terms_per_1000_mean',
    'security_terms_total',
    'share_articles_with_security_terms',
    'afd_far_right_terms_per_1000_mean',
    'afd_far_right_terms_total',
    'share_articles_with_afd_far_right_terms',
    'remigration_democracy_terms_per_1000_mean',
    'remigration_democracy_terms_total',
    'share_articles_with_remigration_democracy_terms',
    'negative_threat_terms_per_1000_mean',
    'negative_threat_terms_total',
    'share_articles_with_negative_threat_terms',
    'security_share_of_migration_context_mean',
    'migration_terms_per_1000_median',
    'security_terms_per_1000_median',
    'afd_far_right_terms_per_1000_median',
    'remigration_democracy_terms_per_1000_median',
    'negative_threat_terms_per_1000_median',
]
text_merge_cols = [c for c in text_merge_cols if c in text_monthly.columns]

merged = combined.merge(
    text_monthly[text_merge_cols],
    on='year_month',
    how='left',
    validate='one_to_one',
)

merged['month_dt'] = pd.to_datetime(merged['year_month'] + '-01',
    errors='coerce')
merged['month'] = merged['year_month']
merged['year'] = merged['month_dt'].dt.year
merged['month_num'] = merged['month_dt'].dt.month

if len(merged) != EXPECTED_MERGED_ROWS:
```

```

        raise ValueError(f'Merged panel row mismatch: got {len(merged)}, expected_{EXPECTED_MERGED_ROWS}')
    if merged['year_month'].duplicated().any():
        raise ValueError('Merged panel has duplicate months')

merged_periods = pd.PeriodIndex(merged['year_month'], freq='M')
missing_merged_months = EXPECTED_MERGED_RANGE.difference(merged_periods)
extra_merged_months = merged_periods.difference(EXPECTED_MERGED_RANGE)
if len(missing_merged_months) > 0 or len(extra_merged_months) > 0:
    raise ValueError(
        'Merged panel coverage mismatch. '
        f'missing={ [p.strftime("%Y-%m") for p in missing_merged_months] }, '
        f'extra={ [p.strftime("%Y-%m") for p in extra_merged_months] }'
    )

for col in ['afd_poll_support_mean', 'nexis_total_results']:
    if col not in merged.columns:
        raise ValueError(f'Merged panel missing required column: {col}')
    if merged[col].isna().any():
        bad = merged.loc[merged[col].isna(), 'year_month'].tolist()[:10]
        raise ValueError(f'Merged panel has missing values in {col}. Sample_{months: {bad}}')

for col in required_text_cols:
    if col not in merged.columns:
        raise ValueError(f'Merged panel missing required text column after_{merge: {col}}')
    if merged[col].isna().any():
        bad = merged.loc[merged[col].isna(), 'year_month'].tolist()[:10]
        raise ValueError(f'Merged panel has missing text values in {col}._{Sample months: {bad}}')

ordered_front = [
    'month', 'year_month', 'month_dt', 'year', 'month_num',
    'polling_source_block', 'collection_method', 'afd_poll_support_mean',
    'nexis_total_results', 'media_salience_volume_raw',
    'media_salience_volume_log1p',
    'media_salience_volume_z', 'media_salience_volume_z_panel',
    'n_articles_text_indicators',
    'migration_terms_per_1000_mean',
    'security_terms_per_1000_mean',
    'afd_far_right_terms_per_1000_mean',
    'remigration_democracy_terms_per_1000_mean',
    'negative_threat_terms_per_1000_mean',
]
ordered_front = [c for c in ordered_front if c in merged.columns]
other_cols = [c for c in merged.columns if c not in ordered_front]

```

```

merged_out = merged[ordered_front + other_cols].sort_values('month_dt').
↳reset_index(drop=True)

merged_out.to_csv(OUT_FULL_PANEL_CSV, index=False)
merged_out.to_parquet(OUT_FULL_PANEL_PARQUET, index=False)

print('Saved merged full panel CSV:', OUT_FULL_PANEL_CSV)
print('Saved merged full panel parquet:', OUT_FULL_PANEL_PARQUET)
print('merged_out shape:', merged_out.shape)
print('coverage:', merged_out['year_month'].min(), 'to',
↳merged_out['year_month'].max())
print('Preview:')
print(merged_out.head(5).to_string(index=False))

```

```

Saved merged full panel CSV: c:\PythonProjects\AfD-
project\data\panel\monthly_full_analysis_panel_2013_2025.csv
Saved merged full panel parquet: c:\PythonProjects\AfD-
project\data\panel\monthly_full_analysis_panel_2013_2025.parquet
merged_out shape: (148, 67)
coverage: 2013-09 to 2025-12
Preview:
  month year_month  month_dt  year  month_num  polling_source_block
collection_method  afd_poll_support_mean  nexis_total_results
media_salience_volume_raw  media_salience_volume_log1p  media_salience_volume_z
media_salience_volume_z_panel  n_articles_text_indicators
migration_terms_per_1000_mean  security_terms_per_1000_mean
afd_far_right_terms_per_1000_mean  remigration_democracy_terms_per_1000_mean
negative_threat_terms_per_1000_mean
notes  afd_poll_support_median  afd_poll_support_sd  n_polls  n_pollsters
first_poll_date  last_poll_date  afd_poll_support_z_combined  raw_article_count
download_fraction  kept_article_count  cleaned_sample_size
cleaned_sample_fraction  retrieval_mismatch_flag  high_download_sample_flag
month_polling  month_dt_polling  year_polling  month_num_polling  source
afd_poll_support_min  afd_poll_support_max  month_media  year_media
month_num_media  cleaned_sample_log1p  cleaned_sample_z  month_dt_media
text_indicator_month_has_articles  mean_word_count  median_word_count
total_word_count  migration_terms_total  share_articles_with_migration_terms
security_terms_total  share_articles_with_security_terms
afd_far_right_terms_total  share_articles_with_afd_far_right_terms
remigration_democracy_terms_total
share_articles_with_remigration_democracy_terms  negative_threat_terms_total
share_articles_with_negative_threat_terms
security_share_of_migration_context_mean  migration_terms_per_1000_median
security_terms_per_1000_median  afd_far_right_terms_per_1000_median
remigration_democracy_terms_per_1000_median
negative_threat_terms_per_1000_median
2013-09    2013-09 2013-09-01  2013          9 historical_manual_wahlrecht

```

manual_wahlrecht_historical_supplement				4.775	
95		95		4.564348	
-0.560585			-0.589327		80
21.745415			3.812320		0.838495
0.344271			3.034788	Pre-DAWUM historical supplement	
for Cologne-period descriptive diagnostics.					4.7
0.150000	4	3	2013-09-22	2013-09-27	
-1.474761		91	0.957895		80
80		0.842105		False	True
2013-09	2013-09-01		2013		9 Wahlrecht.de
4.7		5.0	2013-09	2013	9
4.394449		1.269012	2013-09-01		True
582.100000		501.5		46568	881
1.000000		164			0.500000
36			0.162500		13
0.100000			152		0.687500
0.508709			18.716578		0.295159
0.0				0.0	
2.938283					
2013-10	2013-10	2013-10-01	2013		10 historical_manual_wahlrecht
manual_wahlrecht_historical_supplement					5.125
151		151			5.023881
-0.191453			-0.224905		125
25.383122			6.346638		0.630590
0.506801			3.793329	Pre-DAWUM historical supplement	
for Cologne-period descriptive diagnostics.					5.0
0.629153	4	3	2013-10-10	2013-10-25	
-1.409967		141	0.933775		125
125		0.827815		False	True
2013-10	2013-10-01		2013		10 Wahlrecht.de
4.5		6.0	2013-10	2013	10
4.836282		3.672159	2013-10-01		True
547.040000		491.0		68380	1614
1.000000		360			0.576000
40			0.160000		32
0.176000			273		0.688000
0.411570			23.923445		1.763668
0.0				0.0	
2.631579					
2013-11	2013-11	2013-11-01	2013		11 historical_manual_wahlrecht
manual_wahlrecht_historical_supplement					4.600
146		146			4.990433
-0.224411			-0.257443		117
19.630492			4.365138		0.404572
0.521013			2.996077	Pre-DAWUM historical supplement	
for Cologne-period descriptive diagnostics.					5.0
0.547723	5	3	2013-11-07	2013-11-29	
-1.507159		136	0.931507		117

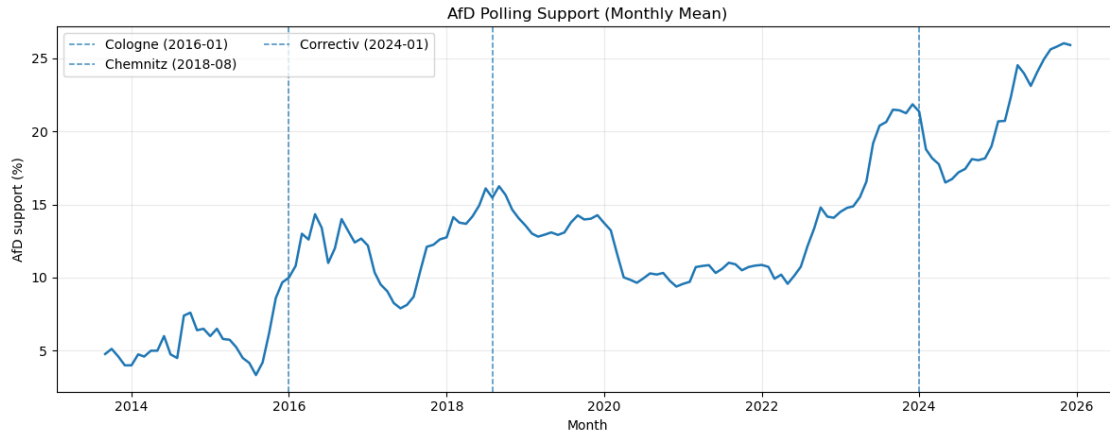
117	0.801370		False	True
2013-11	2013-11-01	2013	11 Wahlrecht.de	
4.0	5.0	2013-11	2013	11
4.770685	3.244933	2013-11-01		True
557.264957	483.0	65200		1197
0.991453	298		0.623932	
32		0.136752		26
0.179487		200		0.649573
0.455178		17.167382		2.087683
0.0		0.0		
2.127660				
2013-12	2013-12	2013-12-01	2013	12 historical_manual_wahlrecht
manual_wahlrecht_historical_supplement				4.000
131		131		4.882802
-0.323286		-0.355056		45
25.832367		3.916191		0.190053
0.203856		3.343139	Pre-DAWUM historical supplement	
for Cologne-period descriptive diagnostics.				4.0
0.000000	4	3	2013-12-05	2013-12-20
-1.618235		50	0.381679	45
45	0.343511		False	False
2013-12	2013-12-01	2013	12 Wahlrecht.de	
4.0	4.0	2013-12	2013	12
3.828641	-0.600102	2013-12-01		True
525.088889	483.0	23629		582
1.000000	95		0.488889	
4		0.088889		4
0.044444		78		0.666667
0.214792		24.351925		0.000000
0.0		0.0		
2.500000				
2014-01	2014-01	2014-01-01	2014	1 historical_manual_wahlrecht
manual_wahlrecht_historical_supplement				4.000
165		165		5.111988
-0.099170		-0.133800		45
22.624274		7.460329		0.958867
0.624914		3.884310	Pre-DAWUM historical supplement	
for Cologne-period descriptive diagnostics.				4.0
0.000000	5	3	2014-01-09	2014-01-31
-1.618235		50	0.303030	45
45	0.272727		False	False
2014-01	2014-01-01	2014	1 Wahlrecht.de	
4.0	4.0	2014-01	2014	1
3.828641	-0.600102	2014-01-01		True
540.488889	425.0	24322		518
1.000000	135		0.577778	
22		0.288889		16
0.200000		89		0.711111

0.429741	21.314387	2.510986
0.0	0.0	
3.129890		

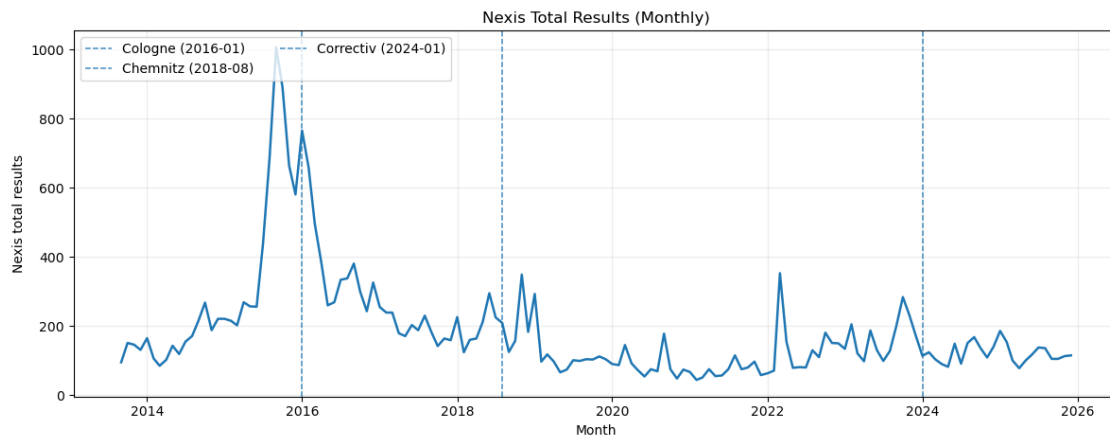
```
[5]: def add_shock_lines(ax):
    for label, dt in SHOCKS.items():
        ax.axvline(dt, linestyle='--', linewidth=1.1, alpha=0.85,
        ↪label=f'{label} ({dt.strftime("%Y-%m")})')

plot_specs = [
    ('afd_poll_support_mean', 'AfD Polling Support (Monthly Mean)', 'AfD_
    ↪support (%)', 'afd_polling_support_mean.png'),
    ('nexis_total_results', 'Nexis Total Results (Monthly)', 'Nexis total_
    ↪results', 'nexis_total_results_monthly.png'),
    ('migration_terms_per_1000_mean', 'Migration Text Intensity (per 1000_
    ↪words, monthly mean)', 'per 1000 words', 'migration_text_intensity_monthly.
    ↪png'),
    ('security_terms_per_1000_mean', 'Security/Public-Order Text Intensity (per_
    ↪1000 words, monthly mean)', 'per 1000 words',
    ↪'security_text_intensity_monthly.png'),
    ('afd_far_right_terms_per_1000_mean', 'AfD/Far-Right Text Intensity (per_
    ↪1000 words, monthly mean)', 'per 1000 words',
    ↪'afd_far_right_text_intensity_monthly.png'),
    ('remigration_democracy_terms_per_1000_mean', 'Remigration/Democracy-Threat_
    ↪Text Intensity (per 1000 words, monthly mean)', 'per 1000 words',
    ↪'remigration_democracy_text_intensity_monthly.png'),
    ('negative_threat_terms_per_1000_mean', 'Negative/Threat Text Intensity_
    ↪(per 1000 words, monthly mean)', 'per 1000 words',
    ↪'negative_threat_text_intensity_monthly.png'),
]

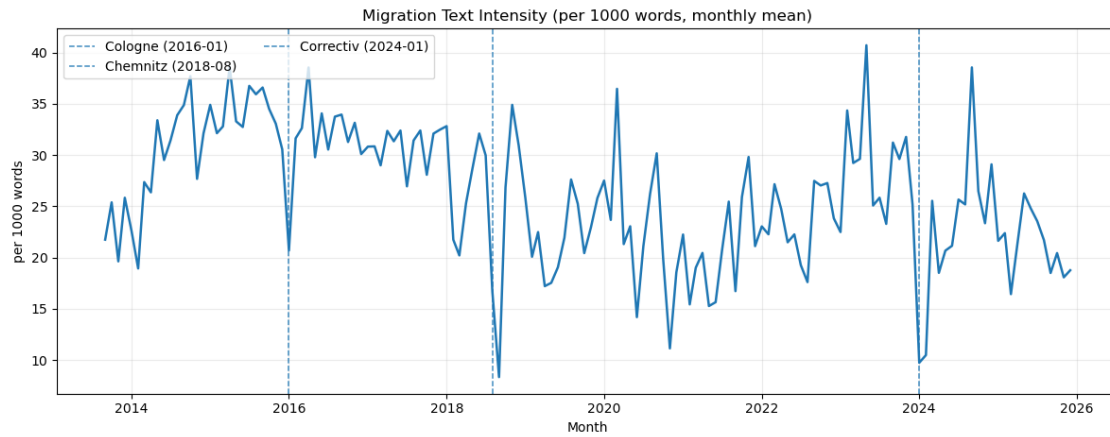
for col, title, ylab, fname in plot_specs:
    fig, ax = plt.subplots(figsize=(12, 4.8))
    ax.plot(merged_out['month_dt'], pd.to_numeric(merged_out[col],
    ↪errors='coerce'), linewidth=1.8)
    add_shock_lines(ax)
    ax.set_title(title)
    ax.set_xlabel('Month')
    ax.set_ylabel(ylab)
    ax.grid(alpha=0.25)
    ax.legend(loc='upper left', ncol=2)
    fig.tight_layout()
    out = FIGURES_DIR / fname
    fig.savefig(out, dpi=150)
    plt.show()
    print('Saved:', out)
```



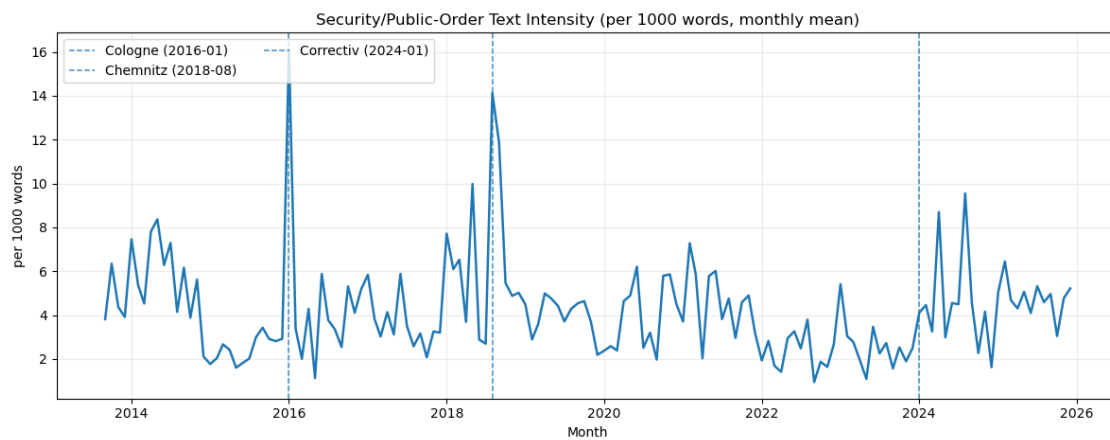
Saved: c:\PythonProjects\AfD-project\figures\08_notebook_figures\afd_polling_support_mean.png



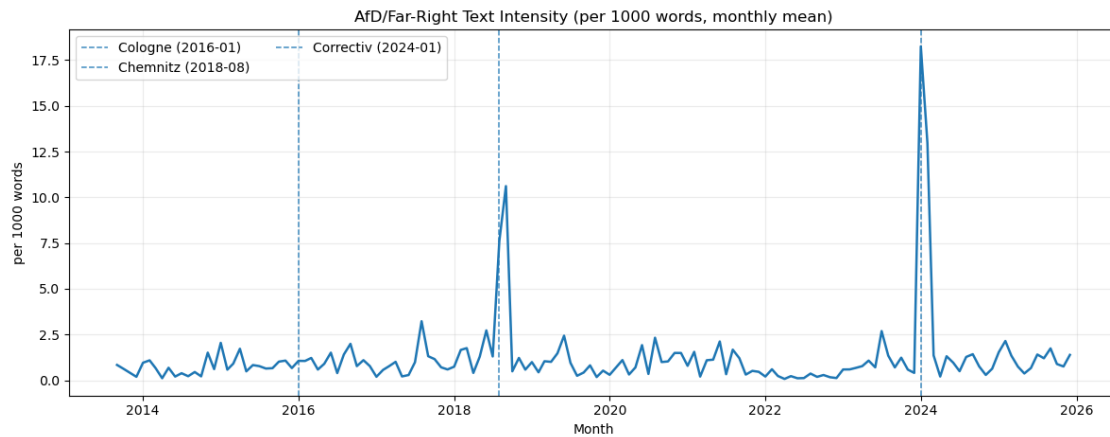
Saved: c:\PythonProjects\AfD-project\figures\08_notebook_figures\nexis_total_results_monthly.png



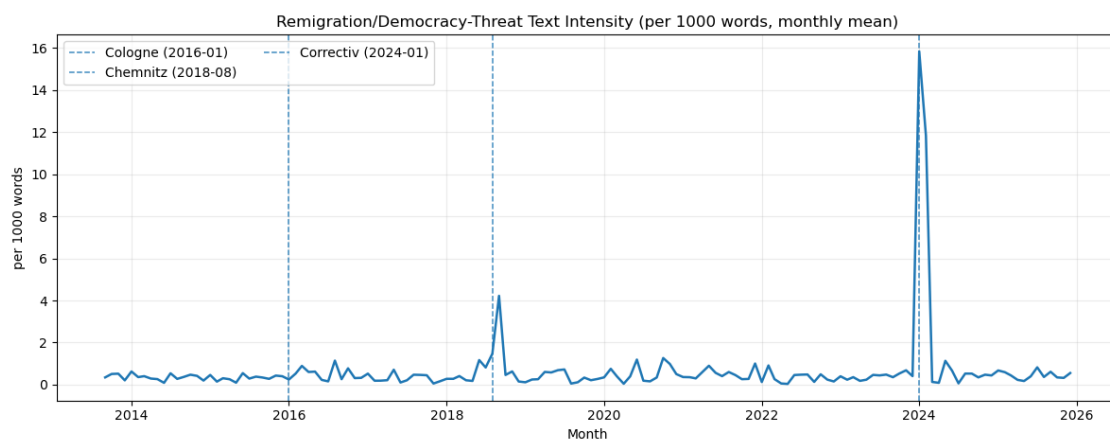
Saved: c:\PythonProjects\AfD-project\figures\08_notebook_figures\migration_text_intensity_monthly.png



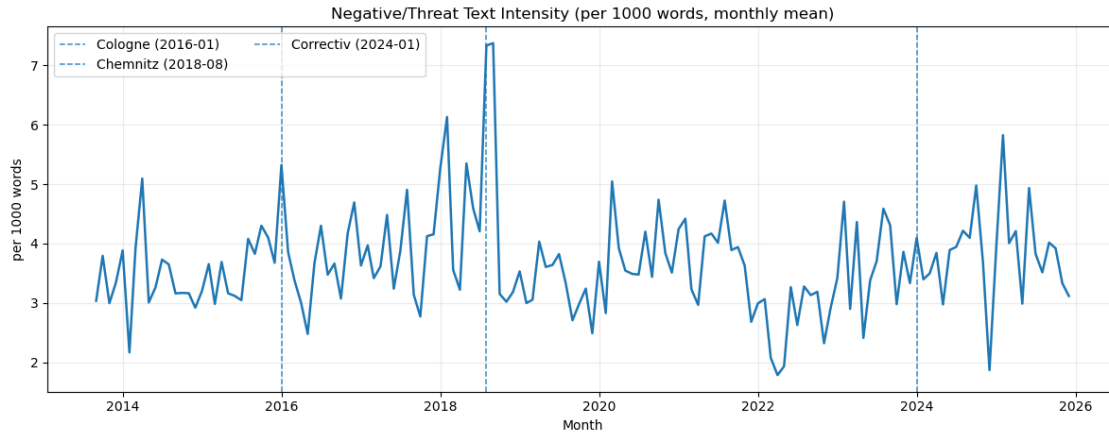
Saved: c:\PythonProjects\AfD-project\figures\08_notebook_figures\security_text_intensity_monthly.png



Saved: c:\PythonProjects\AfD-project\figures\08_notebook_figures\afd_far_right_text_intensity_monthly.png



Saved: c:\PythonProjects\AfD-project\figures\08_notebook_figures\remigration_democracy_text_intensity_monthly.png



Saved: c:\PythonProjects\AfD-project\figures\08_notebook_figures\negative_threat_text_intensity_monthly.png

```
[6]: def zscore(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors='coerce')
    mu = s.mean()
    sd = s.std(ddof=0)
    if pd.isna(sd) or sd == 0:
        return pd.Series(np.zeros(len(s)), index=s.index, dtype=float)
    return (s - mu) / sd

co_cols = [
    'afd_poll_support_mean',
    'nexis_total_results',
    'migration_terms_per_1000_mean',
    'security_terms_per_1000_mean',
    'afd_far_right_terms_per_1000_mean',
    'remigration_democracy_terms_per_1000_mean',
    'negative_threat_terms_per_1000_mean',
]
co = merged_out[['month_dt'] + co_cols].copy()
for c in co_cols:
    co[c + '_z'] = zscore(co[c])

fig, ax = plt.subplots(figsize=(12, 5.2))
color_map = {
    'afd_poll_support_mean_z': '#1f4e79',
    'nexis_total_results_z': '#7a2e2e',
    'migration_terms_per_1000_mean_z': '#2f5d50',
    'security_terms_per_1000_mean_z': '#5f4b8b',
    'afd_far_right_terms_per_1000_mean_z': '#9a4d1f',
    'remigration_democracy_terms_per_1000_mean_z': '#2f7d9d',
}
```

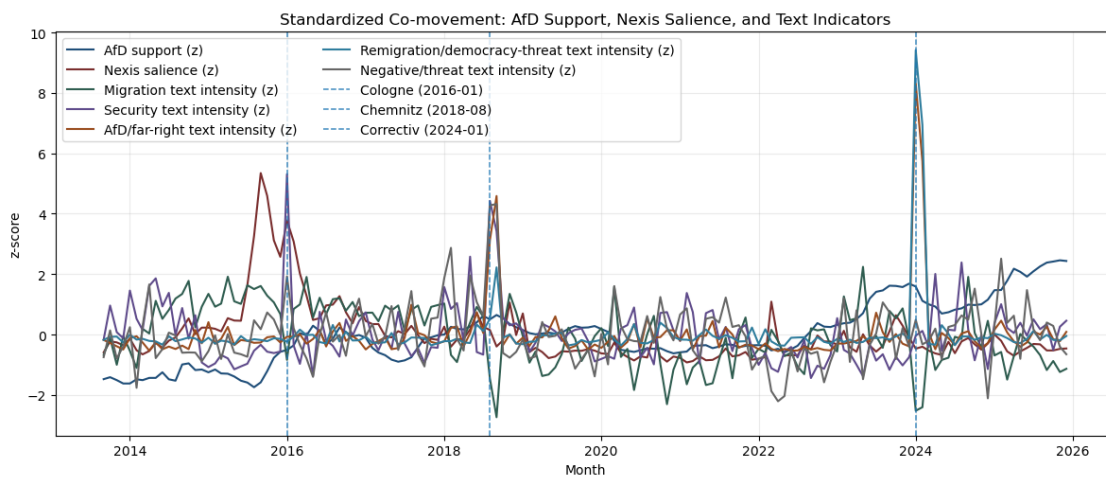
```

        'negative_threat_terms_per_1000_mean_z': '#666666',
    }
    label_map = {
        'afd_poll_support_mean_z': 'AfD support (z)',
        'nexis_total_results_z': 'Nexis salience (z)',
        'migration_terms_per_1000_mean_z': 'Migration text intensity (z)',
        'security_terms_per_1000_mean_z': 'Security text intensity (z)',
        'afd_far_right_terms_per_1000_mean_z': 'AfD/far-right text intensity (z)',
        'remigration_democracy_terms_per_1000_mean_z': 'Remigration/
        ↪ democracy-threat text intensity (z)',
        'negative_threat_terms_per_1000_mean_z': 'Negative/threat text intensity
        ↪ (z)',
    }

    for c in [x + '_z' for x in co_cols]:
        ax.plot(co['month_dt'], co[c], linewidth=1.5, label=label_map[c],
        ↪ color=color_map[c])

    add_shock_lines(ax)
    ax.set_title('Standardized Co-movement: AfD Support, Nexis Salience, and Text
    ↪ Indicators')
    ax.set_xlabel('Month')
    ax.set_ylabel('z-score')
    ax.grid(alpha=0.25)
    ax.legend(loc='upper left', ncol=2)
    fig.tight_layout()
    out = FIGURES_DIR / 'standardized_comovement_full_panel.png'
    fig.savefig(out, dpi=150)
    plt.show()
    print('Saved:', out)

```



Saved: c:\PythonProjects\AfD-
project\figures\08_notebook_figures\standardized_comovement_full_panel.png

```
[7]: summary_vars = ['afd_poll_support_mean'] + KEY_TEXT_COLS[:-1] # keep intensity
      ↪ variables + AfD support
summary_rows = []

for shock_label, shock_month in SHOCKS.items():
    for win in [3, 6]:
        w = merged_out[
            (merged_out['month_dt'] >= shock_month - pd.DateOffset(months=win))
            & (merged_out['month_dt'] <= shock_month + pd.
            ↪ DateOffset(months=win))
        ].copy()
        w['rel_month'] = (
            (w['month_dt'].dt.year - shock_month.year) * 12
            + (w['month_dt'].dt.month - shock_month.month)
        ).astype(int)

        pre = w[w['rel_month'] < 0]
        post = w[w['rel_month'] > 0]
        shock_row = w[w['rel_month'] == 0]

        for var in summary_vars:
            summary_rows.append({
                'shock_label': shock_label,
                'shock_month': shock_month.strftime('%Y-%m'),
                'window_months': win,
                'variable': var,
                'pre_mean': pre[var].mean(),
                'post_mean': post[var].mean(),
                'change_post_minus_pre': post[var].mean() - pre[var].mean(),
                'shock_month_value': shock_row[var].iloc[0] if len(shock_row)
            ↪ == 1 else np.nan,
                'n_months_pre': len(pre),
                'n_months_post': len(post),
            })

shock_summary = pd.DataFrame(summary_rows)
shock_summary.to_csv(OUT_SHOCK_SUMMARY_CSV, index=False)

print('Saved shock-window summary:', OUT_SHOCK_SUMMARY_CSV)
print(shock_summary.head(24).to_string(index=False))

compact =
    ↪ shock_summary[['shock_label', 'shock_month', 'window_months', 'variable', 'change_post_minus_pr
    ↪ copy()
```

```
print()
print('Compact summary (change post-pre):')
print(compact.to_string(index=False))
```

Saved shock-window summary: c:\PythonProjects\AfD-

project\data\panel\full_panel_text_shock_window_summary.csv

shock_label	shock_month	window_months	variable
pre_mean	post_mean	change_post_minus_pre	shock_month_value
n_months_pre	n_months_post		
Cologne	2016-01	3	afd_poll_support_mean
8.155556	12.133333	3.977778	10.000000
3			3
Cologne	2016-01	3	migration_terms_per_1000_mean
32.702058	34.262949	1.560890	20.690986
3			3
Cologne	2016-01	3	security_terms_per_1000_mean
2.882604	3.224113	0.341509	16.113945
3			3
Cologne	2016-01	3	afd_far_right_terms_per_1000_mean
0.921007	0.954998	0.033991	1.061618
3			3
Cologne	2016-01	3	remigration_democracy_terms_per_1000_mean
0.367826	0.671951	0.304125	0.244027
3			3
Cologne	2016-01	3	negative_threat_terms_per_1000_mean
4.026025	3.411755	-0.614270	5.323650
3			3
Cologne	2016-01	6	afd_poll_support_mean
6.027778	12.522222	6.494444	10.000000
6			6
Cologne	2016-01	6	migration_terms_per_1000_mean
34.559518	32.861208	-1.698310	20.690986
6			6
Cologne	2016-01	6	security_terms_per_1000_mean
2.847438	3.409287	0.561849	16.113945
6			6
Cologne	2016-01	6	afd_far_right_terms_per_1000_mean
0.806666	0.944740	0.138074	1.061618
6			6
Cologne	2016-01	6	remigration_democracy_terms_per_1000_mean
0.351435	0.502293	0.150858	0.244027
6			6
Cologne	2016-01	6	negative_threat_terms_per_1000_mean
3.838286	3.446632	-0.391654	5.323650
6			6
Chemnitz	2018-08	3	afd_poll_support_mean
15.070079	15.515632	0.445552	15.463158
3			3

Chemnitz	2018-08	3	migration_terms_per_1000_mean
30.256969	23.366049	-6.890920	16.697127
3			3
Chemnitz	2018-08	3	security_terms_per_1000_mean
5.187018	7.404339	2.217321	14.128618
3			3
Chemnitz	2018-08	3	afd_far_right_terms_per_1000_mean
1.772015	4.108203	2.336187	7.604621
3			3
Chemnitz	2018-08	3	remigration_democracy_terms_per_1000_mean
0.720026	1.769270	1.049244	1.472552
3			3
Chemnitz	2018-08	3	negative_threat_terms_per_1000_mean
4.716825	4.515310	-0.201516	7.337937
3			3
Chemnitz	2018-08	6	afd_poll_support_mean
14.462367	14.529741	0.067374	15.463158
6			6
Chemnitz	2018-08	6	migration_terms_per_1000_mean
26.337535	24.488928	-1.848607	16.697127
6			6
Chemnitz	2018-08	6	security_terms_per_1000_mean
5.312036	5.768046	0.456010	14.128618
6			6
Chemnitz	2018-08	6	afd_far_right_terms_per_1000_mean
1.522423	2.389564	0.867141	7.604621
6			6
Chemnitz	2018-08	6	remigration_democracy_terms_per_1000_mean
0.508479	0.968710	0.460231	1.472552
6			6
Chemnitz	2018-08	6	negative_threat_terms_per_1000_mean
4.510449	3.876131	-0.634319	7.337937
6			6

Compact summary (change post-pre):

shock_label	shock_month	window_months	variable
change_post_minus_pre	shock_month_value		
Cologne	2016-01	3	afd_poll_support_mean
3.977778	10.000000		
Cologne	2016-01	3	migration_terms_per_1000_mean
1.560890	20.690986		
Cologne	2016-01	3	security_terms_per_1000_mean
0.341509	16.113945		
Cologne	2016-01	3	afd_far_right_terms_per_1000_mean
0.033991	1.061618		
Cologne	2016-01	3	remigration_democracy_terms_per_1000_mean
0.304125	0.244027		
Cologne	2016-01	3	negative_threat_terms_per_1000_mean

-0.614270	5.323650		
Cologne	2016-01	6	afd_poll_support_mean
6.494444	10.000000		
Cologne	2016-01	6	migration_terms_per_1000_mean
-1.698310	20.690986		
Cologne	2016-01	6	security_terms_per_1000_mean
0.561849	16.113945		
Cologne	2016-01	6	afd_far_right_terms_per_1000_mean
0.138074	1.061618		
Cologne	2016-01	6	remigration_democracy_terms_per_1000_mean
0.150858	0.244027		
Cologne	2016-01	6	negative_threat_terms_per_1000_mean
-0.391654	5.323650		
Chemnitz	2018-08	3	afd_poll_support_mean
0.445552	15.463158		
Chemnitz	2018-08	3	migration_terms_per_1000_mean
-6.890920	16.697127		
Chemnitz	2018-08	3	security_terms_per_1000_mean
2.217321	14.128618		
Chemnitz	2018-08	3	afd_far_right_terms_per_1000_mean
2.336187	7.604621		
Chemnitz	2018-08	3	remigration_democracy_terms_per_1000_mean
1.049244	1.472552		
Chemnitz	2018-08	3	negative_threat_terms_per_1000_mean
-0.201516	7.337937		
Chemnitz	2018-08	6	afd_poll_support_mean
0.067374	15.463158		
Chemnitz	2018-08	6	migration_terms_per_1000_mean
-1.848607	16.697127		
Chemnitz	2018-08	6	security_terms_per_1000_mean
0.456010	14.128618		
Chemnitz	2018-08	6	afd_far_right_terms_per_1000_mean
0.867141	7.604621		
Chemnitz	2018-08	6	remigration_democracy_terms_per_1000_mean
0.460231	1.472552		
Chemnitz	2018-08	6	negative_threat_terms_per_1000_mean
-0.634319	7.337937		
Correctiv	2024-01	3	afd_poll_support_mean
-3.280122	21.340000		
Correctiv	2024-01	3	migration_terms_per_1000_mean
-10.669344	9.742082		
Correctiv	2024-01	3	security_terms_per_1000_mean
3.162307	4.108073		
Correctiv	2024-01	3	afd_far_right_terms_per_1000_mean
4.103202	18.249246		
Correctiv	2024-01	3	remigration_democracy_terms_per_1000_mean
3.478576	15.842675		
Correctiv	2024-01	3	negative_threat_terms_per_1000_mean

0.187813	4.092657		
Correctiv	2024-01	6	afd_poll_support_mean
-3.653033	21.340000		
Correctiv	2024-01	6	migration_terms_per_1000_mean
-7.476995	9.742082		
Correctiv	2024-01	6	security_terms_per_1000_mean
2.495467	4.108073		
Correctiv	2024-01	6	afd_far_right_terms_per_1000_mean
1.723701	18.249246		
Correctiv	2024-01	6	remigration_democracy_terms_per_1000_mean
1.837664	15.842675		
Correctiv	2024-01	6	negative_threat_terms_per_1000_mean
-0.204342	4.092657		

1.1 Interpretation Notes (Descriptive)

- The merged panel now aligns polling, volume salience, and text-content indicators in a single monthly file for 2013-09 to 2025-12.
- Shock-month windows above are descriptive diagnostics only (local level shifts around predefined dates).
- These results do not establish causal effects; they are intended to guide later modeling choices.

```
[8]: print('Final validation checkpoint:')
print('full panel rows:', len(merged_out))
print('full panel coverage:', merged_out['year_month'].min(), 'to',
      merged_out['year_month'].max())
print('missing AfD support:', int(merged_out['afd_poll_support_mean'].isna().
      sum()))
print('missing Nexis salience:', int(merged_out['nexis_total_results'].isna().
      sum()))
for col in KEY_TEXT_COLS:
    print(f'missing {col}:', int(merged_out[col].isna().sum()))
```

```
Final validation checkpoint:
full panel rows: 148
full panel coverage: 2013-09 to 2025-12
missing AfD support: 0
missing Nexis salience: 0
missing migration_terms_per_1000_mean: 0
missing security_terms_per_1000_mean: 0
missing afd_far_right_terms_per_1000_mean: 0
missing remigration_democracy_terms_per_1000_mean: 0
missing negative_threat_terms_per_1000_mean: 0
missing n_articles_text_indicators: 0
```